

GNexus: Bridging In-Memory and Near-Memory Processing for Density-Aware Graph Analytics

Wenjiong Wang^{1,2,3}, Shuibing He^{1,*}, Siling Yang^{1,*}, Xuechen Zhang⁴,

Ping Chen¹, Tong Wu¹, Gang Chen¹, Yucheng Zhang⁵

¹Zhejiang University

²Zhejiang Key Laboratory of Big Data Intelligent Computing

³Zhejiang Lab, ⁴Kennesaw State University, ⁵Nanchang University

{wenjiongwang, heshuibing, slingzjunet, zjuchenping, wu.tong, cg}@zju.edu.cn
xzhang54@kennesaw.edu, zhangyc_hust@126.com

Abstract

Graph Convolutional Networks (GCNs) face severe memory-wall challenges due to large-scale, irregular graph structures. Although In-Memory Processing (IMP) and Near-Memory Processing (NMP) architectures can reduce data movement, each alone is limited in handling the diverse sparsity and locality patterns in real-world graphs.

This paper introduces GNexus, a hardware–software co-designed GCN accelerator that combines IMP and NMP execution paradigms within a single ReRAM–CMOS die, without requiring monolithic integration of heterogeneous memory technologies such as ReRAM and DRAM. On the hardware side, GNexus uses ReRAM crossbars to provide the IMP computation, while the NMP path is implemented with CMOS logic and SRAM buffers placed near the ReRAM peripheral circuits. A centralized controller and a data bridge buffer enable asynchronous parallelism across the IMP and NMP execution domains without explicit synchronization, while preserving a lightweight and practical implementation. On the software side, GNexus employs a two-level graph partitioning algorithm to match workload characteristics to hardware capabilities, a subgraph-partitioning-based vertex parallelism strategy to increase IMP efficiency, and a latency-model-based vertex reassignment mechanism to maintain runtime load balance. Evaluated across multiple large-scale graph datasets, GNexus achieves average speedups of 8.80× and 5.14×, along with 8.79× and 7.38× energy savings, over state-of-the-art IMP and NMP accelerators, respectively, demonstrating the effectiveness of its hardware–software co-design.

*Corresponding authors.



This work is licensed under a Creative Commons Attribution 4.0 International License.

EuroSys '27, Rabat, Morocco

© 2027 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2971-3/2027/04

<https://doi.org/10.1145/3842654.3848542>

CCS Concepts: • Computer systems organization → Architectures; • Hardware; • Computing methodologies → Machine learning;

Keywords: Processing-In-Memory, Heterogeneous Systems, Parallel Computing, Graph Convolutional Networks

ACM Reference Format:

Wenjiong Wang, Shuibing He, Siling Yang, Xuechen Zhang, Ping Chen, Tong Wu, Gang Chen, and Yucheng Zhang. 2027. GNexus: Bridging In-Memory and Near-Memory Processing for Density-Aware Graph Analytics. In *22nd European Conference on Computer Systems (EuroSys '27)*, April 19–23, 2027, Rabat, Morocco. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3842654.3848542>

1 Introduction

Graph Convolutional Networks (GCNs) have emerged as essential tools for learning over non-Euclidean domains such as molecular structures [9], e-commerce recommendation systems [63], and knowledge graphs [44]. GCNs consist of aggregation and combination operations: aggregation is memory-bound due to irregular sparsity patterns, while combination is compute-bound [51]. Large-scale graph datasets amplify the memory bottleneck, with aggregation dominating runtime on CPUs, GPUs, and accelerators [12, 46, 48, 56].

Memory-centric accelerators including in-memory processing (IMP) [1, 16, 18, 32, 40, 41, 48, 56, 64] and near-memory processing (NMP) [10, 21, 27, 62] aim to address this bottleneck by reducing data movement. However, existing designs rely on homogeneous cores—either IMP or NMP—which limits their capability to fully exploit the heterogeneous locality, density, and connectivity patterns in real-world graphs (Fig. 1(a)). Dense intra-community regions demand compute-efficient in-situ operations, whereas sparse inter-community regions require flexible memory access and computational modes. Neither IMP nor NMP alone provides optimal performance across these diverse patterns.

This work identifies an opportunity to address these limitations by combining the complementary strengths of IMP and NMP. We propose an IMP–NMP accelerator with coordinated hardware architecture and software algorithms to

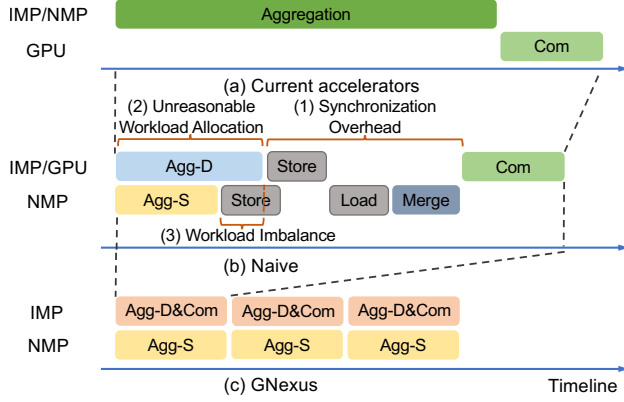


Figure 1. Architecture comparison. “Com” indicates combination; “Agg-D/S” indicates aggregation of dense/sparse graph parts; “Store/Load” indicates storing/loading intermediate results to/from memory; “Merge” indicates merging intermediate results.

enable efficient parallel execution across the two cores. Realizing the full potential of this heterogeneous design requires addressing the following challenges (Fig. 1(b)).

(1) Minimizing synchronization overhead. A naive coupling of IMP and NMP introduces data synchronization overheads that stall compute units and generate substantial additional memory traffic. Moreover, aggregation and combination operations become inherently serialized.

(2) Effective workload allocation. To fully exploit the performance benefits of heterogeneous execution, workloads must be carefully allocated to reduce execution cycles and ineffective computations on the IMP side, while mitigating random memory accesses and computational overhead on the NMP side.

(3) Maximizing workload balance. Heterogeneous hardware characteristics and irregular graph structures can cause significant runtime load imbalance between IMP and NMP cores. This is a challenge that existing GCN accelerators [21, 46, 62] fail to address.

In this paper, we present the GNexus accelerator to address the above challenges (Fig. 1(c)). In hardware, GNexus implements IMP and NMP as two coordinated execution domains on a single ReRAM-CMOS die. The design does not assume that ReRAM and DRAM cells are fabricated in the same bank, thereby avoiding implementation complexity. Instead, the NMP domain is realized with CMOS arithmetic and control logic and SRAM buffering adjacent to the ReRAM peripheral circuits. Meanwhile, GNexus incorporates a centralized controller and a data bridge buffer. The controller enables efficient asynchronous parallelism between the IMP and NMP cores through a dedicated status validation mechanism, eliminating unnecessary wait times. The buffer supports concurrent high-bandwidth access to intermediate results, reducing off-chip memory traffic and preventing write conflicts. In

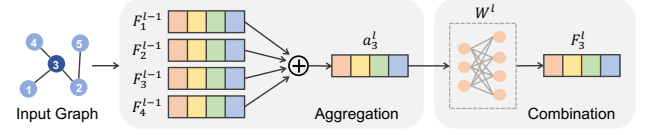


Figure 2. Inference mode of GCN.

software, GNexus first employs a two-level graph partitioning algorithm to assign workloads to the two cores according to their hardware characteristics. The dense parts of subgraphs are computed on the IMP core, while the sparse parts are handled by the NMP core. Given the tasks assigned to the IMP core, GNexus further leverages a subgraph-partitioning-based vertex parallelism strategy, which decomposes each task into subtasks to reduce ineffective computations. Finally, GNexus builds execution time models of both cores. Upon detecting load imbalance, GNexus uses a vertex reassignment strategy to redistribute selected vertices and their associated computations between the two cores.

In summary, we make the following contributions:

- We propose GNexus, a heterogeneous IMP-NMP GCN accelerator that tightly integrates architectural support with software-level execution strategies. GNexus eliminates synchronization overhead between heterogeneous cores and enables efficient parallel execution.
- We develop a two-level graph partitioning algorithm and a subgraph-partitioning-based vertex parallelism strategy that are co-designed with the underlying IMP-NMP hardware. These software techniques maximize the strengths of both core types and ensure that data locality and memory access patterns align with hardware capabilities.
- We introduce a latency-model-based vertex reassignment method that collaboratively leverages hardware characteristics and software scheduling to maintain load balance between the IMP and NMP cores.
- We comprehensively evaluate GNexus on multiple large-scale graph datasets. The results show that for end-to-end GCN inference, GNexus achieves average speedups of 8.80× and 5.14× (and average energy savings of 8.79× and 7.38×) over state-of-the-art IMP and NMP accelerators, respectively.

2 Background

2.1 Graph Convolutional Networks

A GCN consists of multiple graph convolution layers, each composed of two operators: *aggregation* and *combination*. As shown in Fig. 2, the aggregation operation gathers features from neighboring vertices for each vertex, while the combination operation updates vertex features using a multilayer perceptron (MLP). Here, F_v^l is the feature of vertex v at layer l , W^l is the weight matrix of the combination operation at layer l , and a_v^l is the aggregated feature of vertex v . Both operations can be represented as MVM computations [11, 16, 18, 19].

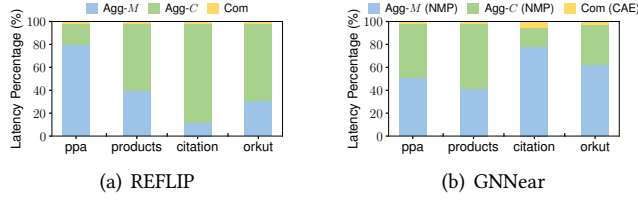


Figure 3. Latency breakdown of different accelerators. M represents memory access, and C represents computation.

Because of the different computation methods, the two compute kernels exhibit different characteristics. The aggregation operation is affected by the graph’s sparsity, making it memory-bound [51]. The combination operation is based on traditional DNNs, making it a compute-bound operation [51].

The graph data used by GCNs typically contains hundreds of millions of vertices and billions of edges, resulting in significant data movement cost [17, 29]. Therefore, the memory-bound aggregation operation tends to become the performance bottleneck of GCNs, accounting for 84% and 94% of the total execution time on the CPU and GPU platforms, respectively [56], resulting in a severe memory wall problem [12, 46, 48].

2.2 Memory-Centric Accelerators

To solve the memory wall problem, memory-centric accelerators have been proposed [1, 10, 13, 16, 18, 21, 32, 34–36, 40, 41, 48, 50, 52, 53, 56, 62, 64]. These accelerators mainly fall into two categories: in-memory processing (IMP) and near-memory processing (NMP). The former, represented by REFLIP [18], processes data in place to minimize data movement overhead. The latter, represented by GNNear [62], reduces data transfer by positioning compute cores near memory units to utilize the internal bandwidth of the memory. Both IMP and NMP are attractive for GCNs because they reduce costly data movement, yet they exploit different strengths. ReRAM-based IMP supports GCNs with highly parallel and latency-efficient in-situ computation [1, 18, 40, 41, 56], while DRAM-based NMP supports GCNs with large memory capacity and easy scalability [5, 62]. Therefore, this study focuses on the complementary execution characteristics of ReRAM-based IMP and DRAM-based NMP, rather than physically integrating these two memory technologies, which would introduce substantial process compatibility, control, and design complexity.

3 Motivation and Challenges

3.1 Motivation

Current memory-centric accelerators primarily use either IMP or NMP cores for aggregation, which remains the dominant bottleneck. Our breakdown analysis shows that aggregation accounts for 97.8% and 96.7% of inference latency in REFLIP [18] and GNNear [62], respectively, across four

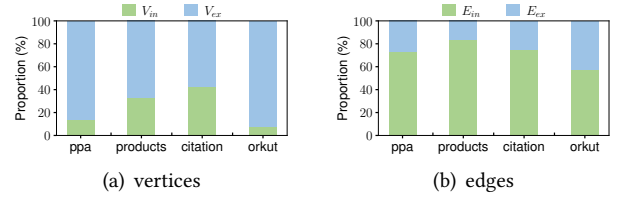


Figure 4. Breakdown of graph structure.

datasets (Fig. 3). We attribute this bottleneck to the combined effects of irregular graph structures and heterogeneous hardware characteristics.

Graph data features. Graph partitioning is a widely used technique in GCN training and inference [6, 12, 22, 48, 55, 60, 61]. Because the size of graph data often exceeds memory capacity, it is necessary to divide a large graph into multiple subgraphs (communities) before executing GCN inference to improve data locality and reduce memory access overhead. We denote the internal edges as E_{in} and the external edges as E_{ex} within a subgraph after graph partitioning. Specifically, E_{in} refers to the edges whose vertices belong to the same subgraph, while E_{ex} refers to the edges that connect vertices across different subgraphs. We use V_{in} and V_{ex} to represent the source vertices of E_{in} and E_{ex} , respectively.

Due to community structure [12, 38], V_{in} typically contains fewer vertices but connects to more edges. Thus, V_{in} requires less memory capacity but higher compute throughput, while V_{ex} exhibits the opposite characteristics. As shown in Fig. 4, on average across four datasets, V_{in} accounts for only 24.3% of source vertices but contributes 72.0% of aggregation edges, whereas V_{ex} accounts for 75.7% of source vertices but contributes only 28.0% of aggregation edges. This contrast between dense intra-community computation and sparse inter-community access makes it difficult for a single compute core to efficiently accelerate aggregation.

Hardware characteristics. (1) *The IMP core is not suitable for processing large and sparse parts of the graph (E_{ex}).* First, the limited capacity of ReRAM-based crossbars prolongs IMP execution when processing a large number of vertices. Crossbar capacity is typically at the megabyte scale, whereas graph data often reaches the gigabyte or even terabyte scale. For example, REFLIP [18] provides only 16 MB of crossbar capacity, whereas the *twitter* [26] dataset exceeds 30 GB. As a result, large vertex features must be repeatedly mapped onto the same crossbars, increasing computation cycles. As shown in Fig. 5, REFLIP requires 7.25× more computation cycles on average than an ideal design with sufficient crossbar capacity to hold all vertex features. Second, sparse graph regions cause numerous ineffective computations on the IMP core. Since more than 99% of the elements in most adjacency matrices are zeros, directly feeding them into crossbar arrays makes over 99% of the crossbar operations ineffective [12, 18, 19].

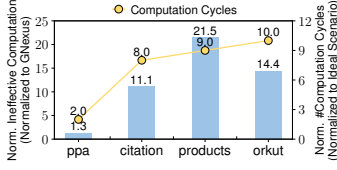


Figure 5. Ineffective computations and computation cycles in REFLIP.

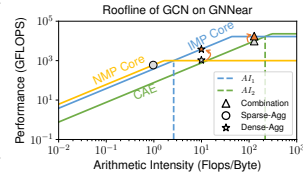


Figure 6. Roofline analysis.

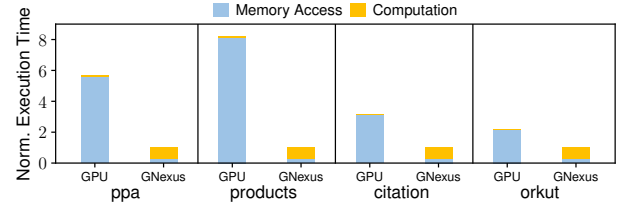


Figure 7. Comparison of execution time of aggregation on H100 GPU and IMP core of GNexus for V_{in} on four datasets.

As a result, REFLIP performs $12.06\times$ as many ineffective computations as GNexus on average (Fig. 5).

(2) *The NMP core is not suitable for processing dense parts of the graph (E_{in}).* First, dense graph regions incur long compute latency on the NMP core. Aggregation workload grows with the average vertex degree, whereas NMP cores typically provide limited compute resources [21, 30, 62]. Even when paired with more powerful auxiliary processors [62], the NMP core can remain the performance bottleneck. Second, dense graph regions also incur long off-chip memory access latency. The NMP core fetches source-vertex features from off-chip memory through highly irregular accesses, so memory latency increases substantially as the average degree grows. GNNear [62], a state-of-the-art heterogeneous NMP accelerator, uses NMP cores for aggregation and a compute-intensive centralized acceleration engine (CAE) for combination. As shown in Fig. 3(b), NMP cores account for 96.7% of GNNear’s total latency, with memory access alone contributing 58.9%, far exceeding the latency of the CAE.

(3) *The compute-intensive cores (e.g., GPUs) incur substantial data movement overhead when executing aggregation, whether processing sparse or dense parts of the graph.* Although V_{in} has a higher arithmetic intensity than V_{ex} , both act as memory-bound operations on compute-intensive cores like the H100 GPU, because their intensities ($\leq 10^1$ Flops/Byte) fall well below the hardware’s threshold ($> 10^2$ Flops/Byte). In contrast, the IMP core is better suited for such tasks due to its in-situ computation capability. Specifically, during aggregation, the IMP core retains subgraph vertex features on-chip to avoid repeated memory accesses. Thus, the IMP core’s memory access volume is proportional to the number of vertices, unlike the GPU’s, which is proportional to the number of edges. This reduction in data movement enables the IMP core to excel in operations with low arithmetic intensity. As shown in Fig. 7, our experiment demonstrates that the IMP core reduces the aggregation latency for V_{in} by an average of $4.82\times$ compared to the H100 GPU. This is because memory access constitutes 98.3% of the total GPU delay, creating a significant bottleneck, whereas it accounts for only 24.6% in the IMP core. Further experimental results in §5.2 and §5.3 reinforce this observation, highlighting the efficiency advantages of IMP over GPUs.

Case study. Based on the above analysis, we conclude that V_{in} can achieve better performance on the IMP core, while V_{ex}

achieves better performance on the NMP core. In Fig. 6, we conduct a roofline analysis of GNNear [62], which illustrates the performance achievable (y-axis) by operations of different arithmetic intensities (x-axis) on various platforms. The red markers represent the optimal allocation strategy, while the gray markers represent the allocation strategy of GNNear. As shown in Fig. 6, the aggregation arithmetic intensity of V_{ex} falls within $[0, AI_1]$, indicating that they perform better on the NMP core. In contrast, the aggregation arithmetic intensity of V_{in} is within $[AI_1, AI_2]$, indicating better performance on the IMP core. A similar analysis applies to combination, so we also execute it on the IMP core.

Therefore, we propose a heterogeneous accelerator to efficiently handle the aggregation of V_{in} and V_{ex} . This accelerator utilizes the in-situ computing capability of the IMP core and the sparse processing capability of the NMP core to solve the memory bottleneck of GCN.

3.2 Challenges

However, this naive heterogeneous accelerator encounters three major challenges.

(1) **Minimizing synchronization overhead.** Intermediate results of aggregation from E_{in} and E_{ex} must be merged before the next stage, causing substantial synchronization overhead in current heterogeneous architectures (Fig. 1(b)). First, a rigid execution dependency between heterogeneous cores forces them to wait for each other until both sides complete their respective tasks, significantly reducing resource utilization. Second, due to uncertainty in task completion time, intermediate results must be stored in memory, resulting in additional memory access overhead. Finally, aggregation and combination must be done strictly in sequence.

(2) **Effective workload allocation.** Workload allocation is critical to exploiting the benefits of both cores. A poor allocation can make heterogeneous execution counterproductive, especially when sparse graph regions are assigned to the IMP core and incur substantial ineffective computations.

(3) **Maximizing workload balance.** Due to the different characteristics of the two cores and irregularity of graph structures, runtime load imbalance is highly prone to occur. For instance, when running GCN on *collab*, *orkut*, *ppa* datasets, the computation latency of the NMP core is $0.13\times$, $2.04\times$, and $2.75\times$ that of the IMP core, respectively (§5.5).

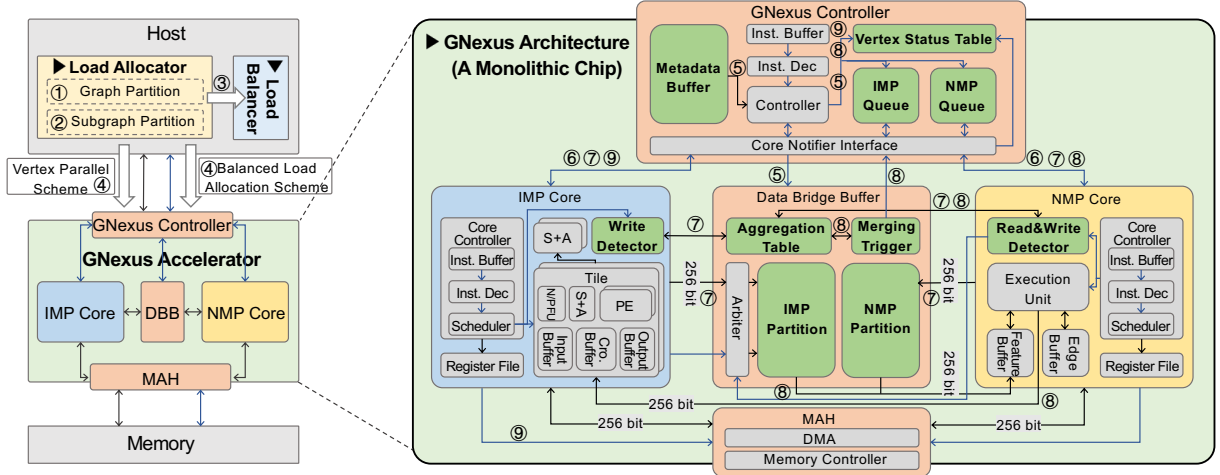


Figure 8. Overview of GNexus accelerator.

4 Design of GNexus

In this paper, we propose GNexus, a hardware–software co-designed accelerator that addresses the aforementioned challenges in large-scale GCN processing. As shown in Fig. 8, GNexus has a hardware platform and two software components. To address the first challenge, we propose the GNexus architecture that minimizes waiting times and memory overhead while overlapping the latency of the combination stage. To address the second challenge, we propose a two-level graph partitioning algorithm and a subgraph-partitioning-based vertex parallelism strategy to enhance the computational efficiency of both cores (Load Allocator). To address the third challenge, we propose a latency-model-based vertex reassignment strategy to balance the load (Load Balancer).

Before inference, the Load Allocator partitions the graph into two subsets tailored to the IMP and NMP cores based on workload and hardware characteristics, and further optimizes the IMP subset to reduce ineffective computations. The Load Balancer then refines the initial allocation to improve load balance across the two cores. This processing is lightweight, accounting for only 7.69% of the end-to-end inference latency in our evaluation (§5.4). Such low overhead allows GNexus to reuse the allocation for static graphs and rerun or update it when graph structures evolve across inference requests (§5.8). Finally, given the finalized allocation, the GNexus accelerator performs inference with the IMP and NMP cores operating asynchronously in parallel.

ReRAM-based IMP and DRAM-based NMP rely on different memory technologies, making bank-level integration difficult to implement. GNexus therefore adopts a unified ReRAM–CMOS die as a more physically realistic solution: ReRAM crossbar arrays are fabricated in BEOL layers above CMOS peripheral circuits, while the NMP functionality is realized using CMOS arithmetic and control

logic and SRAM buffers near the ReRAM banks. This organization avoids physically mixing ReRAM and DRAM cells while following demonstrated ReRAM–CMOS integration approaches [3, 7, 8]. It also keeps cross-domain communication on die through the centralized controller and Data Bridge Buffer (DBB). The resulting hardware overhead remains modest: as shown in Table 2, GNexus occupies 45.56 mm² and consumes 48.72 W, increasing area and power by only 4.38% and 2.82% over REFLIP [18]. Overall, this design stays within a plausible implementation envelope for an architecture-level study.

4.1 Architecture of GNexus

Key idea. As Fig. 8 shows, GNexus mainly comprises five components: the GNexus controller (GC), IMP core, NMP core, DBB, and Memory Access Handler (MAH). The key design concepts are: (1) The GC ensures efficient asynchronous parallelism between the two cores via a status validation mechanism, eliminating waiting and overlapping the latency of combination. (2) The DBB stores intermediate results and supports concurrent accesses under high-bandwidth demands while avoiding write conflicts.

GNexus controller. It serves as the unified scheduling and control center for coordinated computation between IMP and NMP cores, while also functioning as the interface between GNexus and the host. It is responsible for parsing subgraph metadata dispatched by the host and queuing corresponding tasks for the IMP and NMP cores. The controller integrates a metadata buffer (MDB, to store subgraph structural information from the host), a vertex status table (VST, to track the completion status of aggregation and combination for each vertex), and a core notifier interface (to exchange control signals with both cores). During execution, it continuously monitors the execution progress of both cores, coordinating task dispatching and stage transitions. Once the intermediate

results of aggregation are ready, it instructs the NMP core to perform the merge operation, and subsequently signals the IMP core to complete the combination.

IMP core. The IMP core mainly consists of a core controller, shift adders (S+A), a write detector, and multiple tiles, with each tile containing multiple processing elements (PEs), an input buffer (IB), a crossbar buffer (CB), an output buffer, and non-linear and pooling function units for activation. Each PE mainly consists of multiple ReRAM crossbar arrays for in-situ computing. During the combination or aggregation stages, the IMP core maps weight matrices or vertex features onto the crossbar arrays and utilizes vertex features or adjacency matrices as input to perform in-memory computations efficiently. Crucially, the IMP core's in-situ execution fundamentally minimizes data reads. Rather than repeatedly fetching features per edge ($O(|E|)$ reads) as in traditional NMP or GPUs, the IMP core loads them from off-chip memory only once ($O(|V|)$ reads). This completely eliminates the redundant memory accesses and significantly alleviates the memory access bottleneck of GCN. With a typical dimension no larger than 256 [17, 33], the weight matrix occupies only 128KB (assuming a 16-bit data precision), i.e., less than 1% of the IMP core's 16MB memory, thereby avoiding performance degradation in the aggregation. The write detector monitors the *Partial_IMP* status of the corresponding vertex in the aggregation table of the DBB, and initiates a write request and updates the *Partial_IMP* flag only when the target entry is idle and the write conditions are met.

NMP core. The NMP core mainly consists of a core controller, an execution unit (EU), a read & write detector, an edge buffer, and a feature buffer. The EU consists of multiple multipliers, adders, registers, and related control circuits. During aggregation, vertex features and adjacency matrices are loaded into the feature and edge buffers, and dot product operations are then carried out by the EU. Besides checking the readiness of the NMP partition in the DBB, the read & write detector also initiates read requests to the IMP and NMP partitions in the DBB to merge intermediate aggregation results.

Implementing the NMP core in this way reduces data movement while avoiding the complexity of integrating heterogeneous memory technologies. From a data-movement perspective, heterogeneous aggregation requires merging intermediate vectors per vertex. With a conventional DRAM-based NMP engine, such as that in GNNear [62], located off-chip relative to the IMP core, this merging requires intermediate results to traverse DRAM and off-chip links, incurring substantial communication overhead. Bringing both execution domains onto the same chip would reduce this overhead. However, retaining DRAM-based NMP in such an on-chip design would require integrating DRAM and ReRAM, introducing substantial process and design complexity. GNexus therefore realizes the NMP core as a digital aggregation engine built from CMOS logic and SRAM buffers,

allowing it to be integrated with the IMP core and DBB on a single ReRAM-CMOS die.

We refer to this digital aggregation engine as *NMP* in the broader logic-near-memory sense because it processes intermediate results locally near the DBB and IMP core. In the evaluated workloads, this intermediate-result traffic exceeds the off-chip traffic for source-vertex features, making its localization particularly beneficial. Specifically, for each vertex, the engine reads two partial aggregation vectors from the DBB and forwards one merged vector to the IMP core, resulting in a transfer volume three times the size of the original vertex feature vector. This design keeps all three transfers on-chip. The evaluation further shows that GNexus achieves a 6.00 $\times$ speedup over GNNear+Ass, which applies our workload allocation strategy while retaining GNNear's off-chip NMP engines (§5.2).

Data bridge buffer. As a dedicated high-bandwidth, low-latency buffer bridging the IMP and NMP cores, the DBB is constructed from multi-port SRAM to enable concurrent read/write operations. Crucially, the DBB strictly confines the massive intermediate results generated during the aggregation and combination stages to the on-chip SRAM. Conversely, directly applying GNexus's software optimizations to existing architectures (e.g., GNNear [62], HyGCN [51]) would lead to severe off-chip memory access overhead for intermediate results. When IMP writes and NMP reads occur concurrently, the arbiter prioritizes read requests to promptly complete ongoing operations and release buffer space for subsequent tasks. The aggregation table records *Partial_IMP*, *Partial_NMP*, and *Aggregated* bits for each vertex, tracking aggregation progress and managing read/write traffic. The merging trigger monitors these status bits and initiates the merge aggregation command upon detecting completed writes on both sides. To facilitate parallel access, the DBB provides two write ports and one read port, with both cores using the write ports and the NMP core using the read port.

To avoid write conflicts, the DBB is evenly partitioned into two physical SRAM partitions, with each core only writing to its corresponding partition. Since the data stored by both cores is symmetric, no buffer space is wasted. To prevent the DBB becoming a bottleneck, each port's bandwidth should match the data write rate. For a single vertex aggregation, the access latency should be less than the compute latency: $(2 \times D) \times 10^9 / B < \text{compute_latency}$, where the data precision is 16-bit, D is the vertex feature dimension (typically no more than 256 [17, 33]), and B is the bandwidth of the DBB. After load balancing, the compute latencies of the IMP and NMP cores are nearly equal, so we set the IMP core latency (50.88 ns [39]) as *compute_latency*. Therefore, B should be greater than 10.06 GB/s. Since the IMP partition handles both read and write requests concurrently, we configure each port with a bandwidth of 32 GB/s (256-bit at 1GHz).

When the IMP or NMP core produces an intermediate result, it is written to the corresponding partition. Once

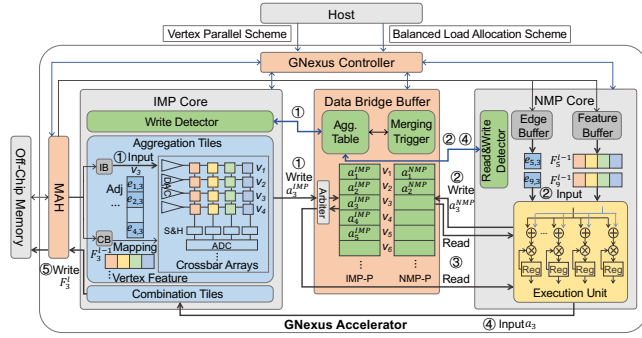


Figure 9. The architecture and workflow of GNexus.

both parts of a vertex's intermediate result are ready, the NMP core aggregates the two parts and inputs them into the combination tiles of the IMP core to complete the combination computation. Taking vertex 3 (v_3) and its source vertices in Fig. 10 as an example, as shown in Fig. 9, the IMP core aggregates the features of v_1 , v_2 , and v_4 , generating the intermediate result a_3^{IMP} (①). Meanwhile, the NMP core aggregates v_5 and v_9 , generating a_3^{NMP} (②). The NMP core reads the two buffers to aggregate a_3^{IMP} and a_3^{NMP} (③), producing a_3 as the input for the IMP core combination tiles (④). After the update is complete, the result F_3^I is written back to the off-chip memory (⑤). See §5.6 for the sensitivity analysis of the DBB size.

Memory access handler. MAH serves as the interface between GNexus and memory, responsible for fetching data. We mask memory access latency through prefetching.

Workflow. We illustrate the workflow for GNexus in Fig. 8. When GNexus receives a GCN inference task, it begins with graph processing on the host. Specifically, the Load Allocator takes task information (model and graph data) and hardware parameters as inputs for graph partitioning to exploit vertex locality and hardware characteristics (①). Each partition is further divided to enhance the compute efficiency of the IMP core (②). The output of the Load Allocator is then fed into the Load Balancer to ensure balanced loads on the two cores (③). Once graph processing is completed, the metadata of the subgraph is written into the GC's MDB, triggering the accelerator to start inference (④). We modify the runtime graph partitioning (clustering) algorithm, islandization [12], to complete the graph partitioning process. It is based on multi-path parallel BFS, starting from high-degree vertices to identify connected communities. It ensures good clustering quality while incurring only a 6.63% additional latency, making it suitable for real-time scenarios (§5.4).

During GCN inference, the GC reads the current subgraph from the MDB, initializes the VST and DBB's aggregation table, and enqueues per-vertex aggregation tasks into the IMP and NMP queues (⑤). The GC then dispatches tasks to each core, which performs its partial aggregation (⑥). Before aggregation begins for each partition, GNexus maps the source

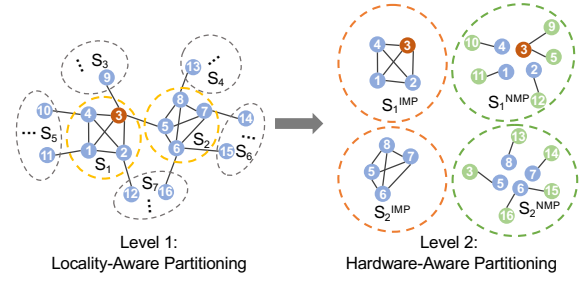


Figure 10. An example of two-level graph partitioning.

vertex features assigned to the IMP core onto the aggregation tiles and reuses them throughout that partition's aggregation. Upon aggregation completion, the detector checks the aggregation table. If $Partial_*$ is 0 or $Aggregated$ is 1, the intermediate result is written to the corresponding partition, the $Partial_*$ flag is set to 1, and the GC is notified to initiate the next task (⑦). The DBB's merging trigger monitors these flags. When both flags are set for a vertex, it signals the GC to enqueue a merging task for the NMP core. The NMP core uses the detector to issue read requests to both partitions, carries out the merge computation, and sets $Aggregated$ to 1, then writes the result to the crossbar buffer of the combination tiles in the IMP core, and notifies the GC (⑧). The GC marks the vertex's aggregation as complete in the VST, issues a combination task to the IMP core, which finishes the final computation, writes it back via the MAH, and signals the GC to set the *Combination* flag (⑨). When all vertices in the subgraph are done, the GC moves on to the next subgraph. The GC and DBB allow the two cores to operate asynchronously, avoiding stalls caused by data dependency. As shown in Fig. 11, the inter-vertex pipeline overlaps combination with aggregation. As the two cores operate in parallel, the IMP core's write latency can also be overlapped by the NMP core's execution time.

4.2 Task Assignment using Two-Level Graph Partitioning

Key idea. Because GNexus integrates both IMP and NMP cores, tasks must be assigned to the appropriate core before hardware mapping to maximize parallelism. Existing graph partitioning algorithms are insufficient because they focus only on data locality and overlook NMP and IMP hardware characteristics. Our partitioning algorithm aims to: (1) exploit data locality, (2) assign tasks to the IMP core to minimize computation cycles, and (3) assign tasks to the NMP core to reduce memory latency. Our approach uses two-level graph partitioning: First, GNexus partitions the graph based on data locality. Second, it further divides each partition into subgraphs based on efficient computation for aggregation on the hardware. Specifically, assume that the input graph is G , which is partitioned into a list of partitions $\{S_1, S_2, \dots, S_n\}$, where $G = S_1 \cup S_2 \cup \dots \cup S_n$. Each partition S_i is further partitioned into S_i^{NMP} and S_i^{IMP} , where $S_i = S_i^{NMP} \cup S_i^{IMP}$.

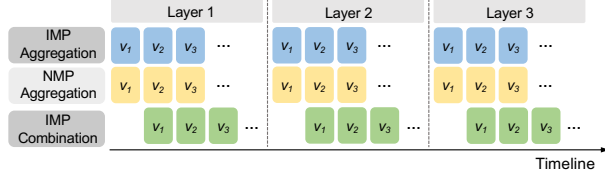


Figure 11. Inter-vertex pipeline of GNexus.

After graph partitioning, GNexus will assign set $\{S_i^{NMP}\}$ to the NMP core and assign set $\{S_i^{IMP}\}$ to the IMP core.

Level 1: locality-aware partitioning. We modify the islandization algorithm [12] to exploit data locality. We use the example in Fig. 10 for illustration. After the locality-aware partitioning, S_1 is composed of $v_1 \sim v_4$ and S_2 is composed of $v_5 \sim v_8$. To reduce feature remapping, we include as many source vertices as possible in each IMP partition without exceeding the IMP core's available crossbar capacity.

Level 2: hardware-aware partitioning. Based on these hardware characteristics discussed in §3.1, at level 2, we further partition S_i into S_i^{IMP} and S_i^{NMP} . S_i^{IMP} and S_i^{NMP} contain all internal and external edges of S_i , respectively. For aggregation within a partition, S_i^{IMP} has fewer source vertices but entails a higher computational load (more edges), which fits the IMP core, while S_i^{NMP} has the opposite characteristics, making it more suitable for the NMP core. For example, S_1^{IMP} contains 4 source vertices and 12 internal edges, while S_1^{NMP} contains 5 source vertices ($v_5, v_9 \sim v_{12}$) but only 5 external edges. As shown in Fig. 10, S_1 is further partitioned into S_1^{IMP} and S_1^{NMP} because $e_{11,1}, e_{12,2}, e_{5,3}, e_{9,3}, e_{10,4}$ are external edges and will be processed by NMP. The remaining internal edges will be aggregated using IMP.

4.3 Subgraph-Partitioning-based Vertex Parallelism Strategy

Key idea. To reduce ineffective computations, we propose the subgraph-partitioning-based vertex parallelism strategy (SPVP). It leverages the inherent locality of graph data through fine-grained partitioning on the IMP core, enabling parallel computation of vertices without external edges in secondary subgraphs. SPVP incurs lower computational overhead than the existing optimization-based algorithm (PASGCN [56]) by exploiting vertex locality in subgraphs. Our experiments show that, compared to PASGCN, SPVP increases the number of parallelizable vertices by $1.97\times$ to $3.41\times$, while reducing the processing time to $0.09\% \sim 2.39\%$ of that required by PASGCN.

For S_i^{IMP} , we implement the following steps. (1) *Subgraph division and classification*: We divide each S_i^{IMP} into n secondary subgraphs $\{SG_1, SG_2, \dots, SG_n\}$ (the sensitivity analysis of n is in §5.6). Then, SG_i is partitioned into two sets: V_i^{in} and V_i^{ex} . The former represents vertices with only internal edges within SG_i , while the latter denotes vertices with both internal and external edges. (2) *Parallelization determination*: We set m as the minimum number of vertices across all

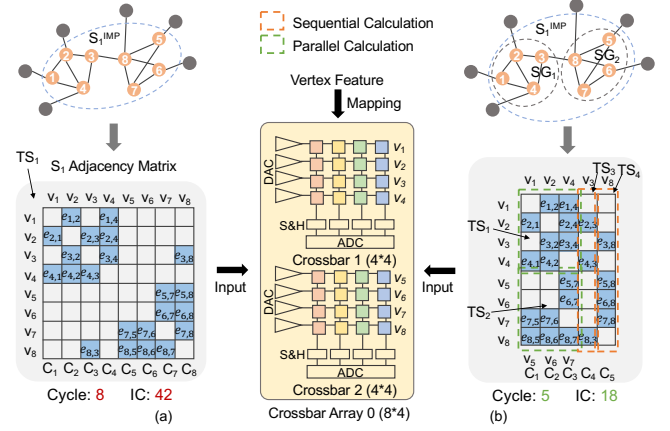


Figure 12. Computation cycles of the graph: (a) Default; (b) After subgraph partitioning.

V_i^{in} . Here, m determines the number of parallel execution rounds. (3) *Parallel computation and serial computation*: For each SG_i , we select m vertices from V_i^{in} based on their degree in descending order to be computed in parallel with other secondary subgraphs. The remaining vertices in each SG_i undergo aggregation serially.

Fig. 12 illustrates our intuition. Suppose S_1^{IMP} is assigned to the IMP core, with its adjacency matrix shown in Fig. 12(a), where each element corresponds to an edge. Crossbar Array 0 has 8 rows, sufficient to map all vertex features in S_1^{IMP} . Aggregation uses the adjacency matrix as input. For instance, in cycle 1 (C_1), feeding the first column enables aggregation of v_2 and v_4 into v_1 via $e_{2,1}$ and $e_{4,1}$. The full aggregation completes in 8 cycles, with 42 ineffective computations (IC).

As shown in Fig. 12(b), when using SPVP, we divide S_1^{IMP} into SG_1 and SG_2 , where SG_1 contains $v_1 \sim v_4$ and SG_2 contains $v_5 \sim v_8$. Here, v_1, v_2, v_4 belong to V_1^{in} and $v_5 \sim v_7$ belong to V_2^{in} . Since $m = 3$, the vertices in V_1^{in} and V_2^{in} can be computed in parallel. Meanwhile, v_3 and v_8 can only be computed serially. After running the algorithm, we obtain four tasks TS_1 to TS_4 . Because TS_1 and TS_2 can be executed in parallel, these two tasks only need 3 cycles to complete, and we can reduce the total computation cycles to 5. The number of ineffective computations also decreases from 42 to 18.

4.4 Latency-Model-based Vertex Reassignment

Key idea. Given $\{S_i^{NMP}\}$ and $\{S_i^{IMP}\}$, IMP and NMP cores may not achieve load balancing, leading to an inefficient pipeline. We introduce a latency-model-based vertex reassignment (LMVR) method, which addresses this challenge by redistributing vertices between $\{S_i^{NMP}\}$ and $\{S_i^{IMP}\}$. Specifically, we create a latency model for both cores, and estimate their execution times $\{T_i^{IMP}\}$ and $\{T_i^{NMP}\}$ for the given sets. If $|T_i^{IMP} - T_i^{NMP}| \geq \text{Watermark}$, the load balancing process is triggered. In addition to computing power and bandwidth, this strategy also considers other hardware factors, including computation patterns, memory access patterns, and memory

Table 1. Parameters used in the execution time models.

Notation	Definition
$T^{IMP/NMP}$	Total execution time of IMP/NMP given $S_i^{IMP/NMP}$
$T_m^{IMP/NMP}$	Memory access latency of IMP/NMP given $S_i^{IMP/NMP}$
$T_c^{IMP/NMP}$	Compute latency of IMP/NMP given $S_i^{IMP/NMP}$
$S_f^{IMP/NMP}$	Memory access amount for vertex features of the IMP/NMP
$S_e^{IMP/NMP}$	Memory access amount for edges of the IMP/NMP
$p^{IMP/NMP}$	Peak throughput of the IMP/NMP
$C^{IMP/NMP}$	Total amount of computation on the IMP/NMP
B_S/B_R	Bandwidth of sequential/random access for off-chip memory
$T_{re}^{IMP/NMP}$	The execution time of IMP/NMP after vertex assignment
$T_\delta^{IMP/NMP}$	The variation in execution time of IMP/NMP
n_x	The number of vertices in the internal partition
n_z	The number of vertices that IMP Core can map
n_m/e_m	The number of vertices/edges that need to be moved
e_y	The number of external edges

capacity. It can balance workloads while incurring only a 1.06% additional overhead (§5.4).

Execution time model of the IMP core. The parameters used in the model are listed in Table 1. The execution time T^{IMP} includes both memory access and compute latency (Equation 1). The memory access latency can be computed as the ratio of the total amount of memory access for retrieving features and edges to the memory bandwidth of sequential access (Equation 2). The compute latency for the IMP core is computed as the ratio of the total amount of computation to the peak throughput of the core (Equation 3).

$$T^{IMP} = T_m^{IMP} + T_c^{IMP} \quad (1)$$

$$T_m^{IMP} = \frac{S_f^{IMP} + S_e^{IMP}}{B_S} \quad (2)$$

$$T_c^{IMP} = \frac{C^{IMP}}{p^{IMP}} \quad (3)$$

Execution time model of the NMP core. Similarly, we can build an execution time model of the NMP core.

$$T^{NMP} = T_m^{NMP} + T_c^{NMP} \quad (4)$$

$$T_m^{NMP} = \frac{S_f^{NMP} + S_e^{NMP}}{B_R} \quad (5)$$

$$T_c^{NMP} = \frac{C^{NMP}}{p^{NMP}} \quad (6)$$

Latency-model-based vertex reassignment. Given the execution time models, we develop an algorithm to adjust $\{S_i^{NMP}\}$ and $\{S_i^{IMP}\}$ when load imbalance occurs.

Specifically, the algorithm first loads subgraph information for each partition. Next, it estimates the execution time of both cores (T^{IMP} and T^{NMP}) using the aforementioned models. Then, if $|T^{IMP} - T^{NMP}| \geq \text{Watermark}$, it calculates the number of vertices to be moved (otherwise, load balancing is not necessary). If $T^{IMP} > T^{NMP}$, we need to reduce the amount of computation on the IMP core without adding too much computation to the NMP core. Since the computation latency of the NMP core is affected by the number of edges connected to vertices, while the IMP core is not, we will move vertices in S_i^{IMP} to S_i^{NMP} and prioritize those with fewer internal edges. If $T^{IMP} < T^{NMP}$, we need to reduce

the amount of computation on the NMP core as much as possible. Thus, we will move vertices in S_i^{NMP} to S_i^{IMP} and prioritize those with more external edges. Finally, the load balanced graph partitions are returned and then mapped to hardware. The sensitivity analysis of *Watermark* is in §5.6.

Calculating the number of vertices for reassignment.

In the vertex reassignment algorithm, we need to estimate the number of vertices to be moved between S_i^{IMP} and S_i^{NMP} (n^m). We can formulate it as an integer optimization problem. Specifically, our goal is to make the execution times of the two cores nearly equal after vertex reassignment. As shown in Equation 7, we define $f(n^m)$ as the execution time difference between the two cores after reassignment. To this end, we model the impact of vertex movement on execution times: for the IMP core, the latency change T_δ^{IMP} is estimated by scaling the original memory and compute latency (T_m^{IMP} , T_c^{IMP}) through the ratio of reassigned vertices n_m to n_x or n_z (Equation 8). The constant 0.5 indicates that when n_m exceeds half of the crossbar capacity, replicas cannot be established to reduce latency. For the NMP core, T_δ^{NMP} is similarly derived using the ratio of reassigned edges e_m to e_y (Equation 9). We use the *SciPy* [45] library in Python to solve this problem, which is a one-time computation.

$$f(n^m) = |T_{re}^{IMP} - T_{re}^{NMP}| \quad (7)$$

$$T_\delta^{IMP} = T_m^{IMP} \times \frac{n_m}{n_x} + T_c^{IMP} \times$$

$$\begin{cases} \lfloor \frac{n_m}{n_z} + 0.5 \rfloor \times \frac{n_m}{n_z}, & \text{if } T^{IMP} > T^{NMP} \\ \max(\lfloor \frac{n_m}{n_z} + 0.5 \rfloor, \frac{n_m}{n_z}), & \text{if } T^{IMP} < T^{NMP} \end{cases} \quad (8)$$

$$T_{re}^{IMP} = \begin{cases} T^{IMP} - T_\delta^{IMP}, & \text{if } T^{IMP} > T^{NMP} \\ T^{IMP} + T_\delta^{IMP}, & \text{if } T^{IMP} < T^{NMP} \end{cases}$$

$$T_\delta^{NMP} = T_m^{NMP} \times \frac{e_m}{e_y} + T_c^{NMP} \times \frac{e_m}{e_y}$$

$$T_{re}^{NMP} = \begin{cases} T^{NMP} + T_\delta^{NMP}, & \text{if } T^{IMP} > T^{NMP} \\ T^{NMP} - T_\delta^{NMP}, & \text{if } T^{IMP} < T^{NMP} \end{cases} \quad (9)$$

5 Evaluation

5.1 Experimental Setup

System settings. We develop a cycle-accurate simulator integrated with Ramulator [24] for evaluation. We model GNexus using a 32 nm process, with its configuration detailed in Table 2. The IMP core, modeled after [18], has 64 tiles, each containing 8 PEs with 32 crossbars of size 64×64 using 2-bit TaOx ReRAM cells (read/write latencies: 29.31 ns / 50.88 ns [39]). The DAC and ADC have 2-bit and 8-bit resolutions, respectively. Buffer energy and area are modeled using CACTI 6.5 [49]. The NMP core follows [62] with an Execution Unit of 128 GOPS. Both cores use INT16 precision, following the configuration adopted in ISAAC [42]. As ReRAM crossbars have limited floating-point support, using a unified integer format across the IMP and NMP cores avoids redundant data-format conversions when intermediate results are exchanged between the two heterogeneous cores. Prior work has shown that such integer computation

Table 2. GNexus configuration. Only major components are listed; chip-level statistics include all components.

Component	Power(mW)	Area(mm2)	Params.	Spec.
PE properties (8 PEs per Tile)				
ADC	64.00	3.84×10^{-2}	Resolution	8 Bits
			Total	32
DAC	8.00	3.40×10^{-4}	Resolution	2 Bits
			Total	32×64
S&H	0.02	8.00×10^{-5}	Total	32×64
Crossbar	6.20	5.10×10^{-4}	Bits per Cell	2
			Size	64×64
			Total	4×8
IR	2.32	3.80×10^{-3}	Size	4KB
OR	0.42	1.40×10^{-3}	Size	512B
S+A	0.80	9.60×10^{-4}	Total	16
Tile properties (64 Tiles per Chip)				
PE Total	654.08	0.36	Total	8
Input Buffer	7.95	0.03	Size	32KB
Crossbar Buffer	59.42	0.21	Size	256KB
Output Buffer	1.28	4.10×10^{-3}	Size	4KB
NFU	2.04	2.40×10^{-3}	Total	8
PFU	3.20	1.90×10^{-3}	Total	8
S+A	0.05	6.00×10^{-5}	Total	1
NMP Core Properties				
Execution Unit	712.40	1.68	Total	1
Feature Buffer	61.23	0.22	Size	128KB
			Total	2
Edge Buffer	2.56	8.20×10^{-3}	Size	4KB
			Total	2
Chip properties				
MAH and GC	580.41	2.65	Total	1
IMP Core	46593.48	39.32	Total	1
NMP Core	1485.59	3.38	Total	1
Data Bridge Buffer	59.42	0.21	Size	256KB
Chip Total	48718.90	45.56	–	–

introduces negligible accuracy loss in GNN inference [14]. In our evaluation, INT16 computation incurs only a 0.71% average accuracy loss compared with the floating-point baseline, confirming its minor impact on model accuracy. REFLIP [18] is modeled similarly to the IMP core in GNexus. Additionally, Ramulator [24] is modified to simulate the NMP core in GNNear [62], and GPGPUSim [2] is extended for GPU and CAE. We configure the system memory as 256GB HBM2e.

Datasets and models. We primarily evaluate GNexus on the ten datasets listed in Table 3. Among them, *mag* [17] and *web* [29] are used to evaluate generality on graphs with weak dense-intra/sparse-inter characteristics (§5.7), while *reddit* [15], *twitter* [26], and *arxiv* are used to evaluate scalability on dense, large-scale, and dynamic graphs, respectively (§5.8). All datasets exceed the IMP core memory of GNexus. We use GCN [25] as the main model and assess GNexus’s generality in §5.7. The GCN comprises three graph convolutional layers with hidden features of dimension 256 [17, 33].

Baselines. For evaluation, we compare GNexus against three types of primary baselines: IMP accelerators, NMP accelerator, and heterogeneous accelerators enhanced by GNexus principles.

- **IMP accelerators.** (1) IMP-Only uses only the IMP core without GCN-specific optimizations, mapping

Table 3. Graph datasets.

	# Vertices	# Edges	Average Vertex Deg.	Dim
<i>arxiv</i>	169,343	1,166,243	6.89	128
<i>reddit</i>	232,965	229,231,784	983.98	602
<i>collab</i>	235,868	2,358,104	10.00	128
<i>ppa</i>	576,289	42,463,862	73.69	58
<i>web</i>	685,230	7,600,595	11.09	128
<i>mag</i>	736,389	10,792,672	14.66	128
<i>products</i>	2,449,029	123,718,280	50.52	100
<i>citation</i>	2,927,963	30,387,995	10.38	128
<i>orkut</i>	3,072,627	117,185,083	38.14	128
<i>twitter</i>	41,652,230	1,468,364,884	35.25	128

vertex features to crossbars and aggregating with the original adjacency matrix. (2) REFLIP is a state-of-the-art ReRAM-based IMP accelerator that employs row-/column-major modes during aggregation for high-/low-degree vertices, to reduce computation cycles.

- **NMP accelerator.** (1) GNNear is a state-of-the-art DRAM-based NMP accelerator. It enhances NMP core performance with the CAE and applies GCN-specific memory access optimizations. We do not include an NMP-Only baseline because using only NMP cores is insufficient for compute-bound combination operations [30]. (2) Since GNexus realizes its NMP domain using a digital aggregation engine, we further incorporate AWB-GCN [11] as an auxiliary digital baseline to validate this hardware design decision. Detailed comparisons are presented in §6.
- **Heterogeneous accelerators.** (1) Naive directly combines IMP and NMP. It places the NMP core on the DRAM memory and uses the IMP core as the main unit. (2) GNNear+Ass combines GNexus’s two-level graph partitioning algorithm with GNNear’s architecture. (3) GPU-GNexus replaces the IMP core in GNexus with an NVIDIA H100 GPU while keeping the software optimization strategies unchanged.

5.2 Overall Performance

We compare the end-to-end inference latency of GNexus across five datasets. Overall, GNexus realizes the minimum latency among all datasets, as shown in Fig. 13.

GNexus vs. IMP accelerators. We have two observations here. (1) Compared to IMP-Only and REFLIP, GNexus improves performance by 54.61× and 8.80× on average, respectively, by reducing computation cycles and ineffective computations. (2) GNexus performs better on large-scale datasets. For example, GNexus improves performance by 17.30× and 5.10× on *collab* compared to IMP-Only and REFLIP, while achieving 136.47× and 15.85× on larger *citation*. It is because they cannot avoid increased computation cycles and ineffective computations, which are particularly severe in large-scale graphs. In contrast, GNexus addresses

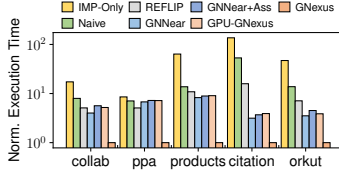


Figure 13. Execution time comparison.

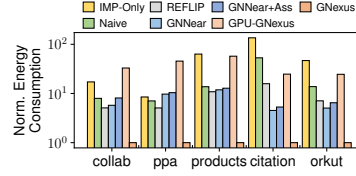


Figure 14. Energy consumption comparison.

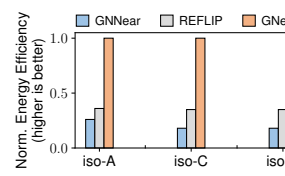


Figure 15. Energy efficiency comparison.

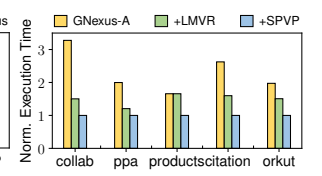


Figure 16. Impact of individual optimization.

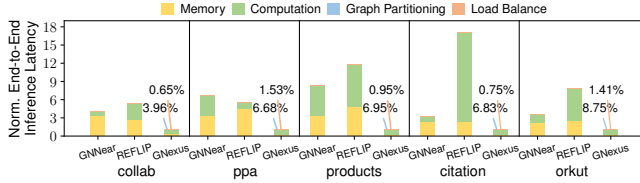


Figure 17. Breakdown of end-to-end inference latency.

this issue through the task assignment and vertex parallel algorithm, as further analyzed in §5.5.

GNexus vs. NMP accelerator. GNexus improves performance by 5.14× on average over GNNear. GNNear relies on NMP cores for aggregation, whose limited compute throughput and irregular memory accesses can become major performance bottlenecks, especially on graphs with high average degrees. In contrast, GNexus addresses these shortcomings through an efficient hardware-software co-design.

GNexus vs. Heterogeneous accelerators. Compared to Naive and GNNear+Ass, GNexus achieves 19.16× and 6.00× speedups, respectively, indicating that simply combining IMP and NMP cores provides limited benefits. These baselines still incur synchronization and intermediate-result movement across heterogeneous execution domains. In contrast, GNexus colocates the NMP core, DBB, and IMP core to keep the merge path on-chip. Compared to GPU-GNexus, GNexus delivers a 5.84× speedup because dense subgraph aggregation remains memory-bound on GPUs. Although GPUs offer higher peak compute capability than IMP cores, irregular memory accesses and excessive data movement lead to low utilization and degraded performance.

5.3 Energy Consumption and Energy Efficiency

We compare the end-to-end inference energy consumption of GNexus and all baselines. As shown in Fig. 14, GNexus reduces energy consumption by 7.38× to 54.27× compared with all baselines. This is because GNexus combines two energy-efficient cores instead of the energy-intensive GPU, while significantly reducing ineffective computations and synchronization costs between hybrid cores.

To ensure a fair comparison, we evaluate the energy efficiency of GNexus, GNNear, and REFLIP under three iso-resource conditions: iso-A (chip area), iso-C (compute throughput), and iso-P (power). As depicted in Fig. 15, GNexus improves energy efficiency by 5.01× and 2.81× over GNNear and REFLIP, respectively.

5.4 Performance Breakdown Analysis

Fig. 17 shows the breakdown of end-to-end inference latency across different workloads. Overall, GNexus substantially reduces both computation and memory latency compared to GNNear and REFLIP. Relative to GNNear, GNexus’s collaborative model compensates for the NMP core’s limited compute capacity and random access pattern, lowering computation and memory latency by 31.28% and 91.38%, respectively. Relative to REFLIP, GNexus eliminates more ineffective computations, reducing computation latency by 73.79%. It also avoids redundant accesses from REFLIP’s column-major execution and overlaps part of the memory latency through efficient parallelism, decreasing memory latency by 91.28%. Although REFLIP also alleviates ineffective computations, its redundant edge loading increases crossbar writes and memory cost, leading to higher latency than GNNear in some cases. GNexus introduces only a negligible 7.69% overhead while achieving 73.18% compute utilization—significantly higher than GNNear (34.61%) and REFLIP (55.55%).

5.5 Impact of Individual Techniques

In this section, we investigate the impact of individual optimizations in GNexus. Fig. 16 shows the execution time of different techniques. GNexus-A represents the use of the GNexus architecture and two-level graph partitioning (§4.1 and §4.2), +LMVR (§4.4) extends the GNexus-A with latency-model-based vertex reassignment, and +SPVP (§4.3) integrates all optimizations. Overall, each optimization applied to the previous version has enhanced performance.

(1) Compared to IMP-Only, GNexus-A achieves a 26.94× speedup by reducing computation cycles by 95.69% (Fig. 18). In IMP-Only, each data mapping requires V cycles (the total number of graph vertices), whereas GNexus leverages graph partitioning to improve data locality, reducing per-mapping cycles to V_s (the number of vertices in each subgraph) while keeping the same number of mappings. Compared to Naive, GNexus-A achieves a 9.02× speedup by removing synchronization overhead, lowering memory access latency by 5.65× and improving compute resource utilization by 22.46%.

(2) Compared to GNexus-A, +LMVR improves performance by 1.56×. This improvement comes from +LMVR’s ability to balance the load between IMP and NMP cores, enabling efficient pipeline execution, as shown in Fig. 19. Without reassignment (Fig. 19(a)), there is a significant execution time

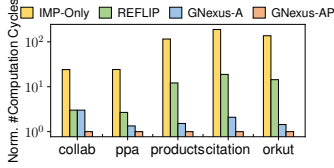
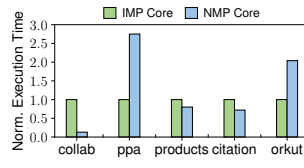
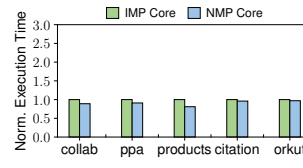


Figure 18. Comparison of IMP core computation cycles. GNexus-AP incorporates SPVP into GNexus-A.



(a) before



(b) after

Figure 19. The execution time ratio of NMP core to IMP before (a) and after (b) load balancing.

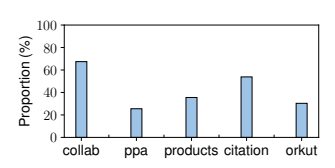


Figure 20. Proportion of parallelizable vertices with the SPVP in GNexus.

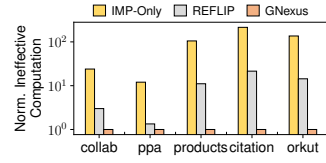


Figure 21. Comparison of ineffective computations generated by different accelerators.

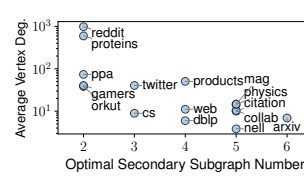


Figure 22. Optimal numbers of secondary subgraphs across datasets with different average vertex degrees.

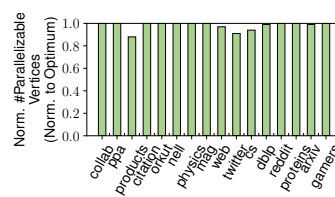


Figure 23. Comparison of parallelizable vertices for default and optimal subgraph partitions.

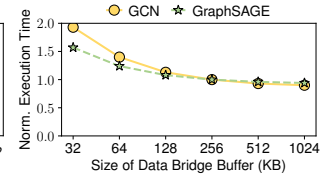


Figure 24. Performance of GNexus with different data bridge buffer sizes.

Table 4. The w range that can trigger load balancing.

Datasets	The Range of w	Datasets	The Range of w
<i>collab</i>	[0, 0.87]	<i>web</i>	[0, 0.90]
<i>ppa</i>	[0, 0.64]	<i>twitter</i>	[0, 0.59]
<i>products</i>	[0, 0.19]	<i>cs</i>	[0, 0.91]
<i>citation</i>	[0, 0.28]	<i>dblp</i>	[0, 0.96]
<i>orkut</i>	[0, 0.51]	<i>reddit</i>	[0, 0.86]
<i>nell</i>	[0, 0.98]	<i>proteins</i>	[0, 0.87]
<i>physics</i>	[0, 0.88]	<i>arxiv</i>	[0, 0.81]
<i>mag</i>	[0, 0.63]	<i>gamers</i>	[0, 0.57]

disparity between the two cores. For instance, on *collab* and *ppa*, the IMP core's execution time is 7.92 $\times$ and 0.36 $\times$ that of the NMP core, respectively. However, with vertex reassignment (Fig. 19(b)), these ratios move closer to 1, reaching 1.13 $\times$ and 1.10 $\times$, respectively. The unchanged performance on *products* is because the load-balancing condition is not triggered.

(3) Compared to +LMVR, +SPVP improves performance by 1.49 $\times$. This is because SPVP increases vertex computation parallelism, reducing ineffective computations. As shown in Fig. 20, 42.53% of the vertices are processed in parallel on average across all datasets. Additionally, ineffective computations in GNexus are only 2.9% of those in IMP-Only and 25.8% of those in REFLIP (Fig. 21). This further reduces computation cycles by 41.77% compared to GNexus-A (Fig. 18).

5.6 Sensitivity Analysis

Number of secondary subgraphs. As shown in Fig. 22, we evaluate the optimal number of secondary subgraphs across 16 graph datasets of different types [17, 29], and find that

the optimal values range from 2 to 6. Among them, 71.43% (5/7) of dense graphs (e.g., *ppa*, *orkut*) perform best with two partitions, while 55.56% (5/9) of sparse graphs (e.g., *collab*, *citation*) achieve optimal performance with five. Guided by these observations, SPVP selects the number of secondary subgraphs independently for each IMP partition according to its sparsity, using two for dense partitions and five for sparse partitions. As shown in Fig. 23, when the optimal value differs from the default, the average loss in parallel vertex count is only 5.28%. Hence, the default configuration provides near-optimal performance, whereas an adaptive strategy would incur higher overhead than benefit.

Size of data bridge buffer. In Fig. 24, we analyze the effect of different DBB sizes on GNexus performance using GCN and GraphSAGE. We observe that GNexus achieves a favorable trade-off between performance and buffer capacity when the DBB size is set to 256KB, as this size balances the computation rates of the IMP core and NMP core without causing excessive buffer waste. Therefore, we set the DBB size to 256KB in this paper.

Watermark. In the load balancing algorithm, the LMVR is triggered if $|T^{IMP} - T^{NMP}| \geq \text{Watermark}$, where $\text{Watermark} = w \times \max(T^{IMP}, T^{NMP})$. A larger w increases the likelihood of the system being recognized as load-balanced. We evaluate the latency ratio between IMP and NMP cores across 16 datasets before applying LMVR and determine the range of w that triggers load balancing, as shown in Table 4. Two key observations are made: (1) Setting $w = 0.2$ ensures that 93.75% of datasets trigger the LMVR strategy and achieve performance improvements. (2) For the remaining datasets, which

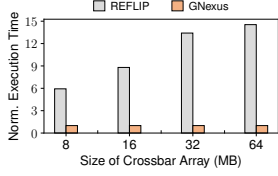


Figure 25. Speedup across different crossbar array sizes.

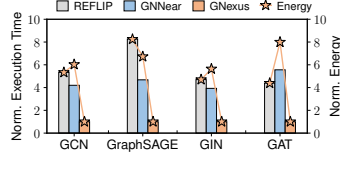


Figure 26. Speedup and energy saving on different models.

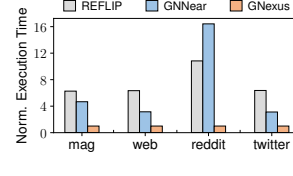


Figure 27. Speedup across diverse and challenging graph structures.

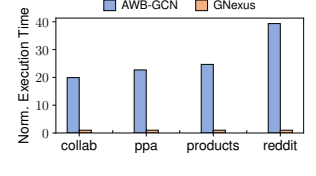


Figure 28. Speedup over AWB-GCN across diverse graph datasets.

are already balanced, applying LMVR may disrupt equilibrium and degrade performance (e.g., on *products*, LMVR activates only when $w \leq 0.19$, leading to a 6.2% slowdown). Considering that the adaptive strategy would introduce additional overhead, we fix $w = 0.2$ in this work.

Size of crossbar array. The capacity of the ReRAM crossbar array affects IMP performance by changing the size and sparsity of the subgraphs mapped to the IMP core. A larger crossbar array can accommodate more vertices, reducing feature remapping and off-chip traffic, but the resulting larger subgraphs may become sparser and reduce crossbar utilization. Our software optimizations adapt to these changes. First, two-level graph partitioning sizes IMP partitions to approach the crossbar capacity limit, reducing data movement. SPVP then selects the number of secondary subgraphs according to each IMP partition’s sparsity, with fewer subgraphs for dense partitions and more for sparse partitions, to reduce ineffective computations. Finally, LMVR adjusts vertex allocation between the IMP and NMP cores based on their workloads to balance their execution times. Together, these strategies maintain consistent performance benefits across crossbar array sizes. As shown in Fig. 25, with crossbar array sizes of 8, 16, 32, and 64 MB, GNexus achieves average speedups of 5.93 $\times$, 8.80 $\times$, 13.41 $\times$, and 14.54 $\times$ over REFLIP, respectively. By efficiently exploiting the complementary capabilities of IMP and NMP, GNexus achieves greater speedups with larger crossbar capacities; however, the additional hardware resources incur greater area and energy costs. We use a 16 MB crossbar array in the other experiments, following the configurations adopted in REFLIP [18].

5.7 Generality Analysis

Different GNN models. We evaluate GNexus on GraphSAGE, GIN, and GAT to verify its generality across GNN models. As shown in Fig. 26, GNexus achieves 4.39 $\times$ –8.26 $\times$ and 3.93 $\times$ –5.56 $\times$ speedups over REFLIP and GNNear, respectively. It also achieves 4.38 $\times$ –8.24 $\times$ and 5.64 $\times$ –7.99 $\times$ energy savings, respectively. Its advantage is most pronounced on GraphSAGE, where sampling-induced sparsity enhances computation parallelism.

Weak dense-intra/sparse-inter pattern. Community structure is a common characteristic of real-world graphs [12, 38]. Nevertheless, certain graphs exhibit a much weaker

dense-intra-community and sparse-inter-community pattern. To evaluate GNexus in such conditions, we further analyze the *mag* [17] and *web* [29] graphs, where this pattern is less pronounced. The proportions of V_{in} in these two graphs are 58.24% and 86.25%, respectively, both exceeding those of V_{ex} and thus deviating from the typical case observed in our other workloads (§3.1). As shown in Fig. 27, GNexus still achieves substantial speedups of 6.31 $\times$ and 3.91 $\times$ over REFLIP and GNNear, respectively, although the gains are lower than those on workloads with a more pronounced dense-intra/sparse-inter pattern. This is because the weaker pattern offers fewer opportunities to exploit the complementary strengths of IMP and NMP. Nevertheless, LMVR helps maintain performance benefits by adjusting vertex allocation to mitigate load imbalance between the two cores. These results highlight both the performance benefits of exploiting the dense-intra/sparse-inter pattern and GNexus’s ability to accommodate graphs where this pattern is less pronounced.

5.8 Scalability Analysis

Larger and denser graphs. To assess GNexus’s scalability, we measure end-to-end inference latency on two representative datasets: *reddit* (high average degree) and *twitter* (large scale), as shown in Fig. 27. GNexus achieves 10.83 $\times$ /16.42 $\times$ speedup over REFLIP/GNNear on *reddit*, and 6.37 $\times$ /3.12 $\times$ on *twitter*. These gains stem from GNexus’s heterogeneous architecture, which efficiently handles diverse graph types. In particular, for dense graphs, GNexus delivers high computational throughput while minimizing redundant operations. Overall, the results demonstrate GNexus’s strong scalability across both large-scale and dense graph datasets.

Dynamic graphs. To evaluate GNexus under evolving graph structures, we construct a dynamic graph from *arxiv* using publication years. Specifically, each snapshot G_t contains all papers published no later than year t , along with citation edges. Consecutive snapshots therefore model graph updates across inference requests, where new papers and citation edges are incrementally added over time. The results show that, although the overhead of graph partitioning and load balancing increases to 18.99% of the inference latency, GNexus still achieves 4.02 $\times$ and 3.39 $\times$ speedups over REFLIP and GNNear, respectively, demonstrating its scalability to dynamic graphs.

6 Discussion

Comparison with digital accelerators. We further compare GNexus with AWB-GCN [11], which improves digital GCN execution through runtime workload rebalancing. As shown in Fig. 28, on *collab*, *ppa*, *products*, and *reddit*, GNexus achieves 19.92 $\times$, 22.69 $\times$, 24.68 $\times$, and 39.36 $\times$ speedups over AWB-GCN, respectively. This improvement mainly stems from GNexus's heterogeneous IMP-NMP architecture, which maps dense and sparse aggregation workloads to their more suitable processing cores and reduces data movement, thereby alleviating the memory-wall bottleneck of conventional digital accelerators.

ReRAM lifetime. ReRAM supports an endurance of 10^{12} write cycles [28]. For one inference, a large graph is divided into subgraphs whose sizes match the storage capacity of the IMP core, requiring multiple remappings of the ReRAM crossbars. Even for our largest workload, *twitter*, the number of remappings per inference is limited to nearly 10^3 . Therefore, under the worst-case assumption that all remappings repeatedly write the same ReRAM cells, the system can still support more than 10^9 inferences before reaching the endurance limit, which is acceptable.

Trade-offs with GPUs. GNexus trades the workload generality of GPUs for higher efficiency on memory-intensive GCN inference. GPUs are better suited for general-purpose workloads, while GNexus targets GCN-centric deployments with tight power, thermal, or space constraints. Its specialization directly addresses the dominant data-movement bottleneck of GCN aggregation. As shown in §3.1, this reduces the aggregation latency of dense graph regions by 4.82 $\times$ compared with an H100 GPU, while the end-to-end evaluation in §5.2 shows a 5.84 $\times$ speedup over GPU-GNexus. Therefore, GNexus should not be viewed as a general replacement for GPUs; rather, it represents a design point that prioritizes efficiency over generality for GCN-centric, resource-constrained deployments.

7 Related Work

GCN accelerators based on FPGA and ASIC platforms. Dedicated accelerators [4, 11, 12, 19, 31, 37, 51, 57, 58] have been designed for FPGA and ASIC platforms. HyGCN [51] separates aggregation and combination into dedicated engines, while AWB-GCN [11] dynamically redistributes sparse workloads across PEs at runtime. GNNIE [37] employs graph-specific caching to reduce irregular off-chip memory accesses. GROW [19] adopts a row-stationary dataflow based on Gustavson's algorithm to improve data reuse. GCoD [58] reshapes graph structures through a tailored training strategy to reduce computational irregularity on dedicated hardware. Overall, these designs improve GCN execution efficiency and data locality, reducing data movement to some

extent. However, they remain constrained by the memory wall of conventional von Neumann architectures.

GCN accelerators based on IMP and NMP. To overcome the memory wall, accelerators based on IMP and NMP architectures [1, 13, 18, 21, 34–36, 40, 41, 43, 47, 56, 59, 62] have been proposed. ReRAM-based IMP accelerators such as REFLIP [18] and CoDG-ReRAM [34] improve GCN execution through optimized crossbar mapping, but highly sparse aggregation still incurs ineffective computations. NMP accelerators like GNNear [62] and gPIM [21] face limitations in computing power and handling large graphs due to random memory access. Unlike these, GNexus uniquely combines NMP and IMP to overcome the limitations of homogeneous cores. Recent works EOD [23] and GoPIM [54] optimize computational graph construction and transmission before GCN inference and replica configuration during GCN training, respectively, and are orthogonal to GNexus.

Heterogeneous execution for GCNs. Existing heterogeneous GCN systems [13, 20, 21, 60, 62] primarily exploit inter-operation heterogeneity by mapping different GCN operations to different execution domains. For example, PyGim [13] executes memory-intensive aggregation on PIM cores and compute-intensive combination on the host processor. However, these designs overlook the heterogeneous computation characteristics within aggregation itself. GNexus further exploits intra-operation heterogeneity by mapping dense and sparse graph regions to complementary IMP and NMP execution domains.

8 Conclusion

This paper presents GNexus, a hardware-software co-designed accelerator for large-scale GCN inference that combines IMP and NMP execution. Its effectiveness comes from three complementary design principles: (1) a lightweight and practical heterogeneous architecture that enables asynchronous parallel execution; (2) software-level task assignment and graph partitioning tailored to the IMP and NMP cores, allowing each core to exploit its strengths; and (3) a latency-model-based load balancing mechanism that redistributes vertices according to predicted execution time. Comprehensive evaluation shows that GNexus achieves significant performance improvements and energy savings.

Acknowledgments

We sincerely thank all the reviewers and our shepherd for their insightful feedback and constructive suggestions. This work was supported by the Joint Funds of the National Natural Science Foundation of China (U25A20423), Major Projects of Zhejiang Province (LD24F020012), Hangzhou Chengxi Sci-Tech Innovation Corridor Special Development Fund (K20241957), Key Research Project of Zhejiang Lab (2025SSYS0005), and the Science and Technology Program of Zhejiang Province (2026SDXT012).

References

- [1] Aqeeb Iqbal Arka, Janardhan Rao Doppa, Partha Pratim Pande, Biresh Kumar Joardar, and Krishnendu Chakrabarty. 2021. ReGraphX: NoC-Enabled 3D Heterogeneous ReRAM Architecture for Training Graph Neural Networks. In *Proceedings of 2021 Design, Automation & Test in Europe Conference & Exhibition*. 1667–1672.
- [2] Ali Bakhoda, George L Yuan, Wilson WL Fung, Henry Wong, and Tor M Aamodt. 2009. Analyzing CUDA Workloads Using a Detailed GPU Simulator. In *2009 IEEE International Symposium on Performance Analysis of Systems and Software*. IEEE, 163–174.
- [3] Bhaswar Chakrabarti, Miguel Angel Lastras-Montaña, Gina Adam, Mirko Prezioso, Brian Hoskins, M Payvand, A Madhavan, A Ghofrani, L Theogarajan, K-T Cheng, et al. 2017. A Multiply-Add Engine with Monolithically Integrated 3D Memristor Crossbar/CMOS Hybrid Circuit. *Scientific Reports* 7, 1 (2017), 42429.
- [4] Cen Chen, Kenli Li, Yangfan Li, and Xiaofeng Zou. 2022. ReGNN: A Redundancy-Eliminated Graph Neural Networks Accelerator. In *Proceedings of 2022 IEEE International Symposium on High-Performance Computer Architecture*. 429–443.
- [5] Dan Chen, Haiheng He, Hai Jin, Long Zheng, Yu Huang, Xinyang Shen, and Xiaofei Liao. 2023. MetaNMP: Leveraging Cartesian-Like Product to Accelerate HGNNs with Near-Memory Processing. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–13.
- [6] Xiaobing Chen, Yuke Wang, Xinfeng Xie, Xing Hu, Abanti Basak, Ling Liang, Mingyu Yan, Lei Deng, Yufei Ding, Zidong Du, et al. 2021. Rubik: A Hierarchical Architecture for Efficient Graph Neural Network Training. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 4 (2021), 936–949.
- [7] Ping Chi, Shuangchen Li, Cong Xu, Tao Zhang, Jishen Zhao, Yongpan Liu, Yu Wang, and Yuan Xie. 2016. PRIME: A Novel Processing-in-Memory Architecture for Neural Network Computation in ReRAM-Based Main Memory. *ACM SIGARCH Computer Architecture News* 44, 3 (2016), 27–39.
- [8] Justin M Correll, Vishishtha Bothra, Fuxi Cai, Yong Lim, Seung Hwan Lee, Seungjong Lee, Wei D Lu, Zhengya Zhang, and Michael P Flynn. 2020. A Fully Integrated Reprogrammable CMOS-RRAM Compute-in-Memory Coprocessor for Neuromorphic Applications. *IEEE Journal on Exploratory Solid-State Computational Devices and Circuits* 6, 1 (2020), 36–44.
- [9] Alex Fout, Jonathon Byrd, Basir Shariat, and Asa Ben-Hur. 2017. Protein Interface Prediction Using Graph Convolutional Networks. *Advances in Neural Information Processing Systems* 30 (2017).
- [10] Daichi Fujiki, Xiaowei Wang, Arun Subramaniyan, and Reetuparna Das. 2021. *In-/Near-Memory Computing*. Springer.
- [11] Tong Geng, Ang Li, Runbin Shi, Chunshu Wu, Tianqi Wang, Yanfei Li, Pouya Haghi, Antonino Tumeo, Shuai Che, Steve Reinhardt, et al. 2020. AWB-GCN: A Graph Convolutional Network Accelerator with Runtime Workload Rebalancing. In *Proceedings of 2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture*. 922–936.
- [12] Tong Geng, Chunshu Wu, Yongan Zhang, Cheng Tan, Chenhao Xie, Haoran You, Martin Herbordt, Yingyan Lin, and Ang Li. 2021. I-GCN: A Graph Convolutional Network Accelerator with Runtime Locality Enhancement through Islandization. In *Proceedings of MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1051–1063.
- [13] Christina Giannoula, Peiming Yang, Ivan Fernandez, Jiacheng Yang, Sankeerth Durvasula, Yu Xin Li, Mohammad Sadrosadati, Juan Gomez Luna, Onur Mutlu, and Gennady Pekhimenko. 2024. PyGim: An Efficient Graph Neural Network Library for Real Processing-in-Memory Architectures. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* 8, 3 (2024), 1–36.
- [14] Christina Giannoula, Peiming Yang, Ivan Fernandez Vega, Jiacheng Yang, Yu Xin Li, Juan Gomez Luna, Mohammad Sadrosadati, Onur Mutlu, and Gennady Pekhimenko. 2024. Accelerating Graph Neural Networks on Real Processing-in-Memory Systems. *arXiv preprint arXiv:2402.16731* 26 (2024).
- [15] Will Hamilton, Zhitaoying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. *Advances in Neural Information Processing Systems* (2017).
- [16] Yintao He, Ying Wang, Cheng Liu, Huawei Li, and Xiaowei Li. 2021. TARE: Task-Adaptive In-Situ ReRAM Computing for Graph Learning. In *2021 58th ACM/IEEE Design Automation Conference*. 577–582.
- [17] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *arXiv preprint arXiv:2005.00687* (2020).
- [18] Yu Huang, Long Zheng, Pengcheng Yao, Qinggang Wang, Xiaofei Liao, Hai Jin, and Jingling Xue. 2022. Accelerating Graph Convolutional Networks Using Crossbar-Based Processing-in-Memory Architectures. In *Proceedings of 2022 IEEE International Symposium on High-Performance Computer Architecture*. 1029–1042.
- [19] Ranggi Hwang, Minhoo Kang, Jiwon Lee, Dongyun Kam, Youngjoo Lee, and Minsoo Rhu. 2023. GROW: A Row-Stationary Sparse-Dense GEMM Accelerator for Memory-Efficient Graph Convolutional Neural Networks. In *Proceedings of 2023 IEEE International Symposium on High-Performance Computer Architecture*. 42–55.
- [20] Seung-Eon Hwang, Duyeong Song, and Jongsun Park. 2024. HeNCoG: A Heterogeneous Near-Memory Computing Architecture for Energy Efficient GCN Acceleration. In *2024 IEEE International Symposium on Circuits and Systems*. IEEE, 1–5.
- [21] Hai Jin, Dan Chen, Long Zheng, Yu Huang, Pengcheng Yao, Jin Zhao, Xiaofei Liao, and Wenbin Jiang. 2023. Accelerating Graph Convolutional Networks through a PIM-Accelerated Approach. *IEEE Trans. Comput.* (2023), 2628–2640.
- [22] George Karypis and Vipin Kumar. 1998. A Fast and High Quality Multilevel Scheme for Partitioning Irregular Graphs. *SIAM Journal on Scientific Computing* (1998), 359–392.
- [23] Taehwan Kim, Yunki Han, Seohye Ha, Jiwan Kim, and Lee-Sup Kim. 2025. EOD: Enabling Low Latency GNN Inference via Near-Memory Concatenate Aggregation. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture*. 1125–1139.
- [24] Yoongu Kim, Weikun Yang, and Onur Mutlu. 2015. Ramulator: A Fast and Extensible DRAM Simulator. *IEEE Computer Architecture Letters* (2015), 45–49.
- [25] Thomas N Kipf and Max Welling. 2016. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv preprint arXiv:1609.02907* (2016).
- [26] Haewoon Kwak, Changhyun Lee, Hosung Park, and Sue Moon. 2010. What Is Twitter, a Social Network or a News Media?. In *WWW '10: Proceedings of the 19th International Conference on World Wide Web* (Raleigh, North Carolina, USA). ACM, New York, NY, USA, 591–600. doi:10.1145/1772690.1772751
- [27] Young-Cheon Kwon, Suk Han Lee, Jaehoon Lee, Sang-Hyuk Kwon, Je Min Ryu, Jong-Pil Son, O Seongil, Hak-Soo Yu, Haesuk Lee, Soo Young Kim, et al. 2021. 25.4 A 20nm 6GB Function-in-Memory DRAM, Based on HBM2 with a 1.2 TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications. In *Proceedings of 2021 IEEE International Solid-State Circuits Conference*. 350–352.
- [28] Mario Lanza, H-S Philip Wong, Eric Pop, Daniele Ielmini, Dimitri Strukov, Brian C Regan, Luca Larcher, Marco A Villena, J Joshua Yang, Ludovic Goux, et al. 2019. Recommended Methods to Study Resistive Switching Devices. *Advanced Electronic Materials* 5, 1 (2019), 1800143.
- [29] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. <http://snap.stanford.edu/data>.
- [30] Cong Li, Zhe Zhou, Yang Wang, Fan Yang, Ting Cao, Mao Yang, Yun Liang, and Guangyu Sun. 2024. PIM-DL: Expanding the Applicability of

- Commodity DRAM-PIMs for Deep Learning via Algorithm-System Co-Optimization. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 879–896.
- [31] Jiajun Li, Ahmed Louri, Avinash Karanth, and Razvan Bunescu. 2021. GCNAX: A Flexible and Energy-Efficient Accelerator for Graph Convolutional Neural Networks. In *Proceedings of 2021 IEEE International Symposium on High-Performance Computer Architecture*. 775–788.
- [32] Cong Liu, Haikun Liu, Hai Jin, Xiaofei Liao, Yu Zhang, Zhuohui Duan, Jiahong Xu, and Huize Li. 2022. ReGNN: A ReRAM-Based Heterogeneous Architecture for General Graph Neural Networks. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 469–474.
- [33] Gongxu Luo, Jianxin Li, Jianlin Su, Hao Peng, Carl Yang, Lichao Sun, Philip S Yu, and Lifang He. 2021. Graph Entropy Guided Node Embedding Dimension Selection for Graph Neural Networks. *arXiv preprint arXiv:2105.03178* (2021).
- [34] Yixuan Luo, Payman Behnam, Kiran Thorat, Zhuo Liu, Hongwu Peng, Shaoyi Huang, Shu Zhou, Omer Khan, Alexey Tumanov, Caiwen Ding, et al. 2022. CoDG-ReRAM: An Algorithm-Hardware Co-Design to Accelerate Semi-Structured GNNs on ReRAM. In *2022 IEEE 40th International Conference on Computer Design*. IEEE, 280–289.
- [35] Yufei Ma, Yikan Qiu, Wentao Zhao, Guoxiang Li, Meng Wu, Tianyu Jia, Le Ye, and Ru Huang. 2024. DCIM-GCN: Digital Computing-in-Memory Accelerator for Graph Convolutional Network. *IEEE Transactions on Circuits and Systems I: Regular Papers* 71, 6 (2024), 2735–2748.
- [36] Sumit K Mandal, Gokul Krishnan, A Alper Goksoy, Gopikrishnan Ravindran Nair, Yu Cao, and Umüt Y Ogras. 2022. COIN: Communication-Aware In-Memory Acceleration for Graph Convolutional Networks. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 12, 2 (2022), 472–485.
- [37] Sudipta Mondal, Susmita Dey Manasi, Kishor Kunal, Ramprasad S, and Sachin S Sapatnekar. 2022. GNNIE: GNN Inference Engine with Load-Balancing and Graph-Specific Caching. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*. 565–570.
- [38] Anurag Mukkara, Nathan Beckmann, Maleen Abeydeera, Xiaosong Ma, and Daniel Sanchez. 2018. Exploiting Locality in Graph Analytics through Hardware-Accelerated Traversal Scheduling. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture*. IEEE, 1–14.
- [39] Dimin Niu, Cong Xu, Naveen Muralimanohar, Norman P Jouppi, and Yuan Xie. 2013. Design of Cross-Point Metal-Oxide ReRAM Emphasizing Reliability and Cost. In *Proceedings of 2013 IEEE/ACM International Conference on Computer-Aided Design*. 17–23.
- [40] Chukwufumnanya Ogbogu, Aqeeb Iqbal Arka, Bireesh Kumar Joardar, Janardhan Rao Doppa, Hai Li, Krishnendu Chakrabarty, and Partha Pratim Pande. 2022. Accelerating Large-Scale Graph Neural Network Training on Crossbar Diet. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 3626–3637.
- [41] Chukwufumnanya O Ogbogu, Aqeeb Iqbal Arka, Lukas Pfromm, Bireesh Kumar Joardar, Janardhan Rao Doppa, Krishnendu Chakrabarty, and Partha Pratim Pande. 2022. Accelerating Graph Neural Network Training on ReRAM-Based PIM Architectures via Graph and Model Pruning. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 8 (2022), 2703–2716.
- [42] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R Stanley Williams, and Vivek Srikumar. 2016. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. *ACM SIGARCH Computer Architecture News* (2016), 14–26.
- [43] Siddhartha Raman Sundara Raman, Lizy John, and Jaydeep P Kulkarni. 2024. NEM-GNN: DAC/ADC-Less, Scalable, Reconfigurable, Graph and Sparsity-Aware Near-Memory Accelerator for Graph Neural Networks. *ACM Transactions on Architecture and Code Optimization* 21, 2 (2024), 1–26.
- [44] Koji Ueno, Toyotaro Suzumura, Naoya Maruyama, Katsuki Fujisawa, and Satoshi Matsuoka. 2016. Extreme Scale Breadth-First Search on Supercomputers. In *Proceedings of 2016 IEEE International Conference on Big Data*. 1040–1047.
- [45] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* (2020), 261–272.
- [46] Yuke Wang, Boyuan Feng, Gushu Li, Shuangchen Li, Lei Deng, Yuan Xie, and Yufei Ding. 2021. GNNAdvisor: An Adaptive and Efficient Runtime System for GNN Acceleration on GPUs. In *Proceedings of 15th USENIX Symposium on Operating Systems Design and Implementation*. 515–531.
- [47] Yiqi Wang, Zihan Wu, Weiwei Wu, Leibo Liu, Yang Hu, Shaojun Wei, Fengbin Tu, and Shouyi Yin. 2024. TensorCIM: Digital Computing-in-Memory Tensor Processor with Multichip-Module-Based Architecture for Beyond-NN Acceleration. *IEEE Journal of Solid-State Circuits* 60, 2 (2024), 734–747.
- [48] Yuntao Wei, Xueyan Wang, Shangdong Zhang, Jianlei Yang, Xiaotao Jia, Zhaohao Wang, Gang Qu, and Weisheng Zhao. 2023. IMGA: Efficient In-Memory Graph Convolution Network Aggregation with Data Flow Optimizations. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2023), 4695–4705.
- [49] Steven JE Wilton and Norman P Jouppi. 1996. CACTI: An Enhanced Cache Access and Cycle Time Model. *IEEE Journal of Solid-State Circuits* (1996), 677–688.
- [50] Tong Wu, Shuibing He, Weijian Chen, Xuechen Zhang, and Tuo Shi. 2026. SIFMAP: Parallel CNN Inference Using Sub-Input-Feature Maps in PIM. *IEEE Trans. Comput.* (2026).
- [51] Mingyu Yan, Lei Deng, Xing Hu, Ling Liang, Yujing Feng, Xiaochun Ye, Zhimin Zhang, Dongrui Fan, and Yuan Xie. 2020. HyGCN: A GCN Accelerator with Hybrid Architecture. In *Proceedings of 2020 IEEE International Symposium on High Performance Computer Architecture*. 15–29.
- [52] Siling Yang, Weijian Chen, Xuechen Zhang, Shuibing He, Yanlong Yin, and Xian-He Sun. 2021. AUTO-PRUNE: Automated DNN Pruning and Mapping for ReRAM-Based Accelerator. In *Proceedings of the 35th ACM International Conference on Supercomputing*. 304–315.
- [53] Siling Yang, Shuibing He, Hexiao Duan, Weijian Chen, Xuechen Zhang, Tong Wu, and Yanlong Yin. 2023. APQ: Automated DNN Pruning and Quantization for ReRAM-Based Accelerators. *IEEE Transactions on Parallel and Distributed Systems* 34, 9 (2023), 2498–2511.
- [54] Siling Yang, Shuibing He, Wenjiong Wang, Yanlong Yin, Tong Wu, Weijian Chen, Xuechen Zhang, Xian-He Sun, and Dan Feng. 2025. GoPIM: GCN-Oriented Pipeline Optimization for PIM Accelerators. In *2025 IEEE International Symposium on High Performance Computer Architecture*. IEEE, 657–670.
- [55] Shuangyan Yang, Minjia Zhang, Wenqian Dong, and Dong Li. 2023. Betty: Enabling Large-Scale GNN Training with Batch-Level Graph Partitioning. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 103–117.
- [56] Tao Yang, Dongyue Li, Fei Ma, Zhuoran Song, Yilong Zhao, Jiaxi Zhang, Fangxin Liu, and Li Jiang. 2022. PASGCN: An ReRAM-Based PIM Design for GCN with Adaptively Sparsified Graphs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* (2022), 150–163.

- [57] Mingi Yoo, Jaeyong Song, Jounghoo Lee, Namhyung Kim, Youngsok Kim, and Jinho Lee. 2023. SGCN: Exploiting Compressed-Sparse Features in Deep Graph Convolutional Network Accelerators. In *2023 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 1–14.
- [58] Haoran You, Tong Geng, Yongan Zhang, Ang Li, and Yingyan Lin. 2022. GCoD: Graph Convolutional Network Acceleration via Dedicated Algorithm and Accelerator Co-Design. In *2022 IEEE International Symposium on High-Performance Computer Architecture*. IEEE, 460–474.
- [59] Sungmin Yun, Hwayong Nam, Jaehyun Park, Byeongho Kim, Jung Ho Ahn, and Eojin Lee. 2023. GraNDe: Efficient Near-Data Processing Architecture for Graph Neural Networks. *IEEE Trans. Comput.* 73, 10 (2023), 2391–2404.
- [60] Hanqing Zeng and Viktor Prasanna. 2020. GraphACT: Accelerating GCN Training on CPU-FPGA Heterogeneous Platforms. In *Proceedings of the 2020 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 255–265.
- [61] Jie-Fang Zhang and Zhengya Zhang. 2021. Point-X: A Spatial-Locality-Aware Architecture for Energy-Efficient Graph-Based Point-Cloud Deep Learning. In *Proceedings of MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 1078–1090.
- [62] Zhe Zhou, Cong Li, Xuechao Wei, Xiaoyang Wang, and Guangyu Sun. 2022. GNNear: Accelerating Full-Batch Training of Graph Neural Networks with Near-Memory Processing. In *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques*. 54–68.
- [63] Rong Zhu, Kun Zhao, Hongxia Yang, Wei Lin, Chang Zhou, Baole Ai, Yong Li, and Jingren Zhou. 2019. AliGraph: A Comprehensive Graph Neural Network Platform. *arXiv preprint arXiv:1902.08730* (2019).
- [64] Yu Zhu, Zhenhua Zhu, Guohao Dai, Kai Zhong, Huazhong Yang, and Yu Wang. 2022. Exploiting Parallelism with Vertex-Clustering in Processing-in-Memory-Based GCN Accelerators. In *Proceedings of 2022 Design, Automation & Test in Europe Conference & Exhibition*. 652–657.